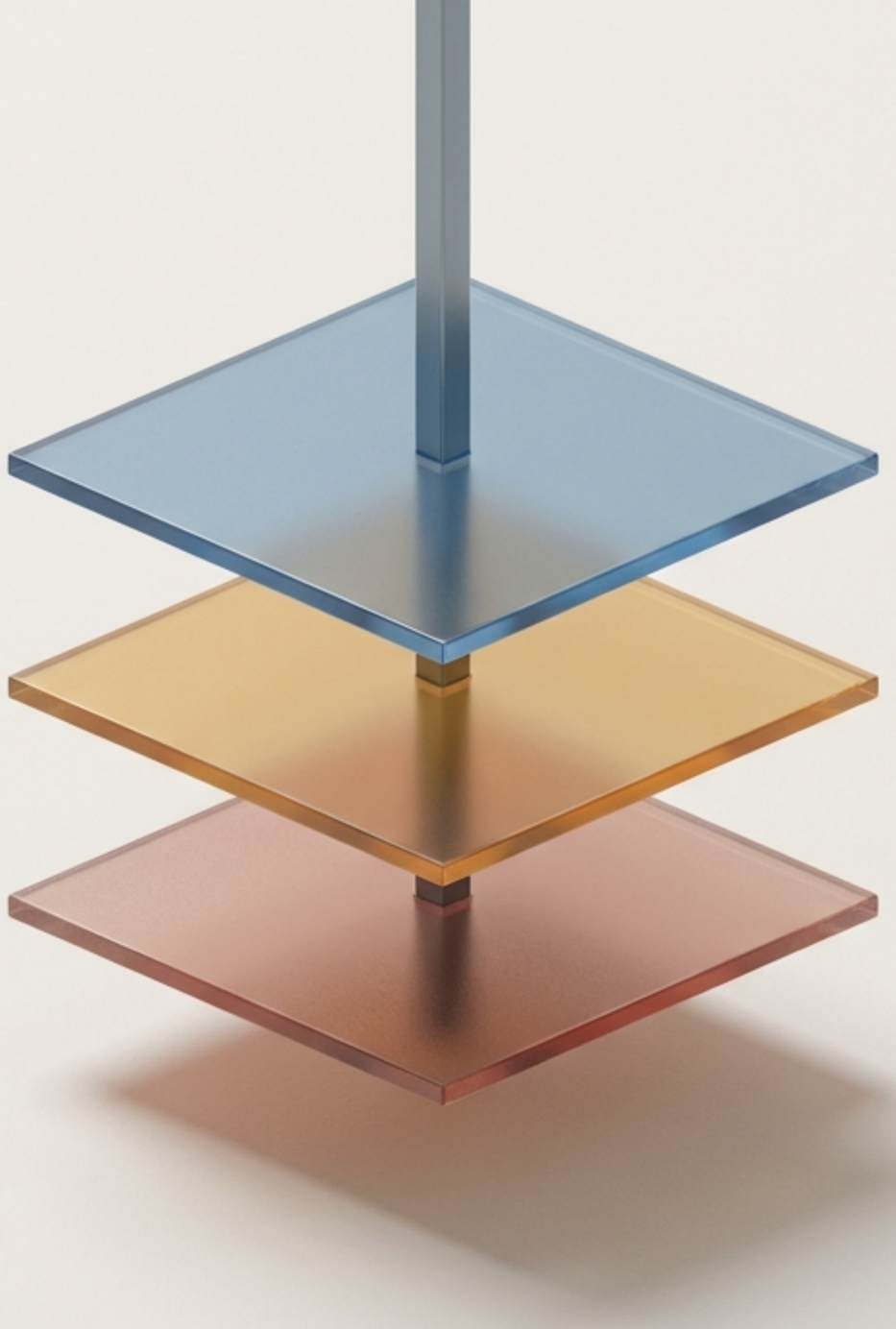
Gradient Boosting for Classification

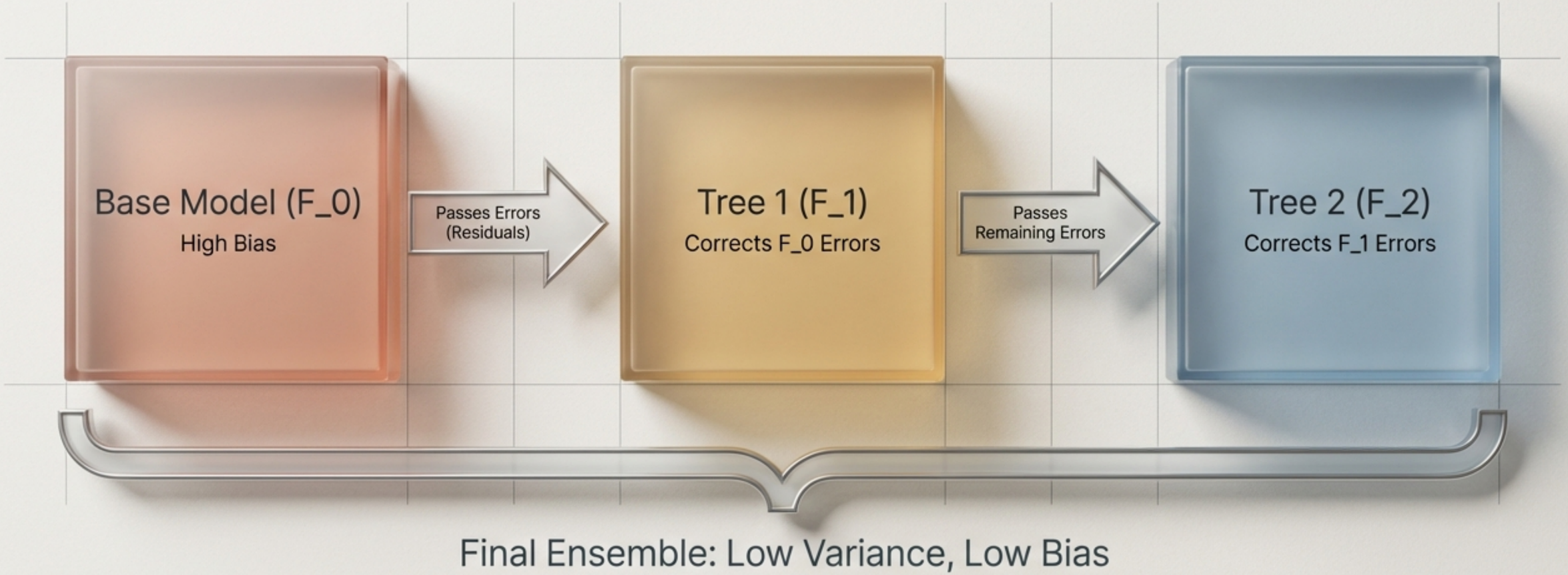
Unlocking the mathematics and geometric intuition under the hood



NotebookLM

Building a Strong Learner from Weak Foundations

The core mechanism relies on **Additive Modelling**. The algorithm trains a sequence of simple, weak models in a stage-wise fashion. Each new layer exists solely to correct the mathematical mistakes of the layer beneath it.



NotebookLM

Same Algorithm, Different Mathematical Engine

The algorithmic sequence remains identical. The fundamental shift occurs entirely within the mathematical engine driving the loss function.

Dimension	Regression	Classification
Goal	Predict continuous value	Predict probability of a class
Loss Function	Mean Squared Error (MSE) $H_i = \sum_{i=1}^{n} (x_i{}^2 - \bar{x}_i)^2$	Log Loss $1 = 1 - \log\left(\frac{1}{x_i+1}\right)$
Initial Model (F_0)	Mean of Target $F_0 = x_i(+k)$	Log-Odds of Target $\texttt{Geist Mono}\ \left(\frac{d}{2v_j+1}\right)$
Tree Type Used	Regression Tree	Regression Tree

Counter-intuitive but true: Classification boosting still uses Regression Trees because it predicts continuous error values, not distinct class labels.

NotebookLM

The Translation Layer: Log-Odds and Probability

Probabilities are bounded between 0 and 1, meaning we cannot safely add them together infinitely. The algorithm solves this by translating everything into **Log-Odds** for additive calculations, converting back to Probability only for evaluation.

The Calculation Space

$$\text{Log-Odds} = \ln\left(\frac{P}{1 - P}\right)$$

Unbounded numeric space where additive trees operate.

The Outcome Space

$$P = \frac{e^{(\text{Log-Odds})}}{1 + e^{(\text{Log-Odds})}}$$

Bounded space (0 to 1) used to evaluate the final prediction.

NotebookLM

The Experimental Blueprint

To observe the engine in motion, we will process a micro-dataset. The objective is to predict whether a student will be placed (1) or not placed (0) based on their academic metrics.

Student ID	CGPA	IQ	Target: Placed
1	7.2	110	1
2	8.1	105	1
3	6.9	98	0
4	7.5	115	1
5	6.5	90	0
6	8.5	120	1
7	7.0	100	0
8	7.8	112	1

Total Dataset Summary:

5 Positive Cases (Placed = 1)
3 Negative Cases (Not Placed = 0)

NotebookLM

Stage 1: Establishing the Naive Baseline (F_0)

The algorithm begins by ignoring all individual student features. It calculates a single, static base prediction for the entire dataset using the initial Log-Odds.

Step 1: Count

5 Placed vs 3 Not Placed

Step 2: Log-Odds Calc

$$\ln\left(\frac{5}{3}\right) = 0.51$$

Step 3: Probability Conversion

Passing 0.51 through the probability formula yields 0.625 (or 62.5%)

Our Stage 1 model predicts a flat 62.5% chance of placement for every student.
It is highly biased, but provides the mathematical foundation.

NotebookLM

Stage 1: Measuring the Baseline's Mistakes

To improve, the model must quantify exactly how **wrong its initial prediction** is for **every** single student. This difference between the actual outcome and the predicted probability is called a **Pseudo-Residual.**

CGPA	IQ	Target	Predicted (p)	Pseudo-Residual
7.2	110	1	0.625	+0.375
8.1	105	1	0.625	+0.375
6.9	98	0	0.625	-0.625
7.5	115	1	0.625	+0.375
6.5	90	0	0.625	-0.625
8.5	120	1	0.625	+0.375
7.0	100	0	0.625	-0.625
7.8	112	1	0.625	+0.375

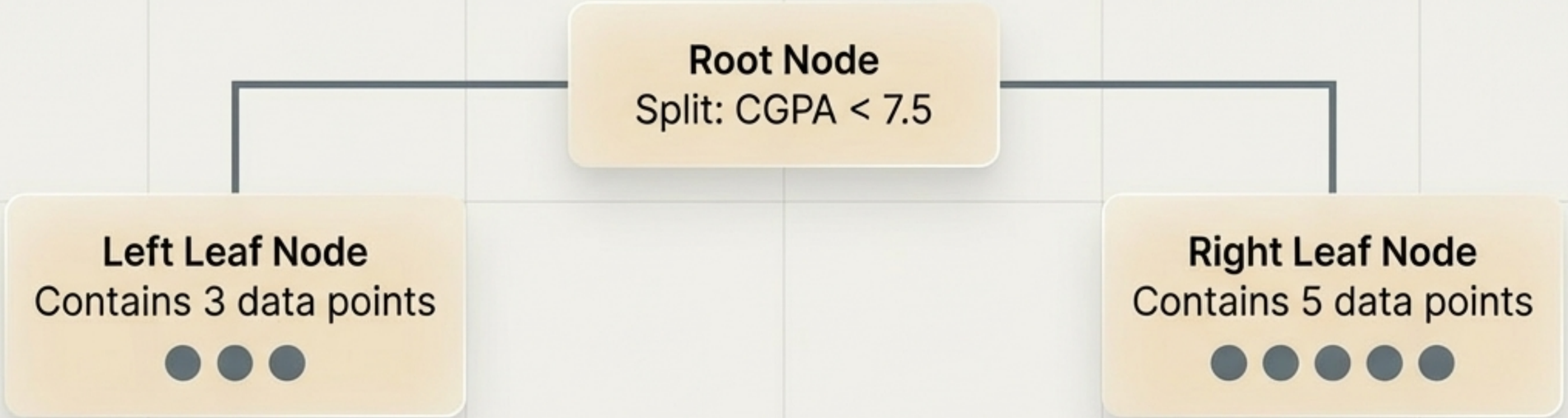
These errors become the specific **Target Variable** for the next stage's decision tree.

```
Actual (y) - Predicted (p)
= Residual
```

NotebookLM

Stage 2: Training the First Weak Learner (F_1)

We construct a restricted, shallow decision tree using CGPA and IQ. However, this tree does not predict classes. It predicts the **Pseudo-Residuals** we just calculated.



Critical Friction Point: This tree outputs residual averages, which are differences in probabilities. We cannot legally add probabilities to our base Log-Odds. We require a mathematical transformation.

NotebookLM

Stage 2: The Leaf Value Transformation (γ)

To align the output scales, each leaf applies a specific transformation formula. This converts the raw probability residuals into a Log-Odds value that can be safely added to the base model.

Formula Deconstruction

$$\gamma = \frac{\text{Sum of Residuals in the leaf (e.g., } +0.375 + -0.625)}{\text{Sum of } [\, P * (1 - P) \,] \text{ for points in the leaf (e.g., } 0.625 * 0.375)}$$

Numerator

Denominator

Outcome Box

$\gamma = -2.1$

This -2.1 is the transformed, additive-ready value for any student falling into this specific leaf.

NotebookLM

The Additive Update and the Learning Rate (\α)

We now combine the Base Model with Tree 1. To prevent the model from over-correcting and memorising noise (**high variance**), we scale the new tree's output by a Learning Rate, typically 0.1.

The result is converted back into a new Probability. The model has now slightly shifted its prediction, taking a small, safe step toward the true value.

NotebookLM

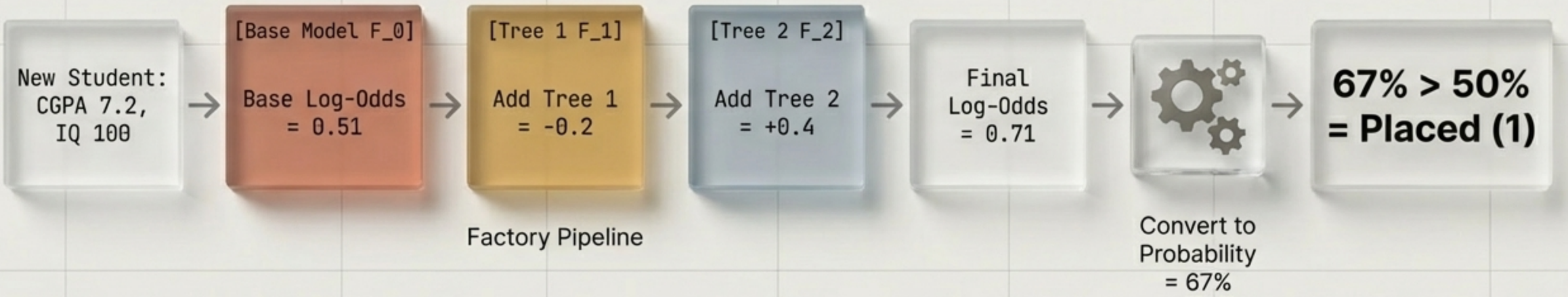
Stage 3 & Beyond: The Iterative Path to Zero Error

As Stage 3 (F_2) and subsequent stages are appended to the chain, the model continuously learns from the remaining mistakes. The absolute size of the pseudo-residuals systematically compresses toward zero.

NotebookLM

The Inference Pipeline: Predicting New Data

A new student profile passes through the entire ensemble sequentially, gathering incremental Log-Odds adjustments before a final probability conversion.

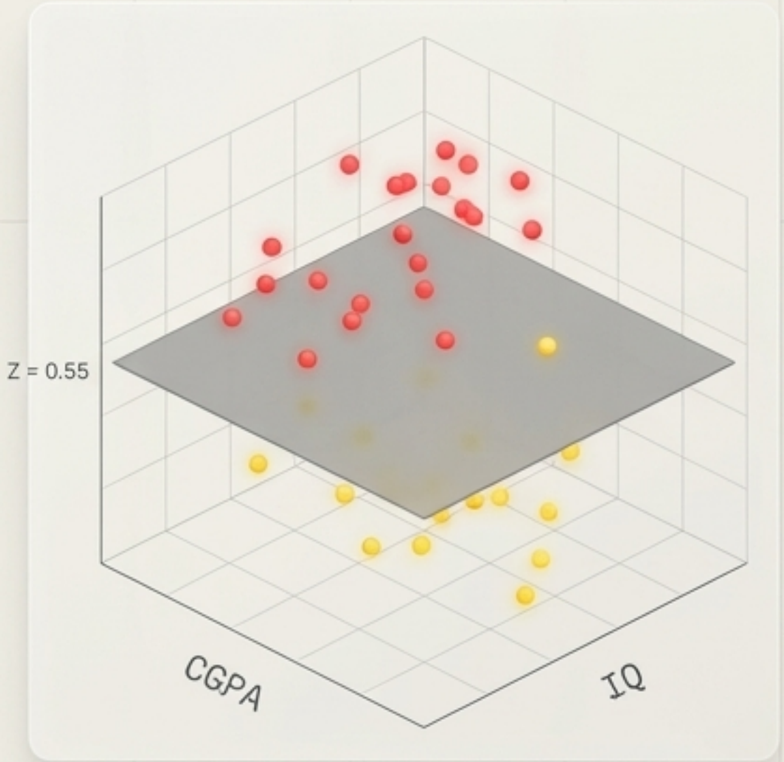


NotebookLM

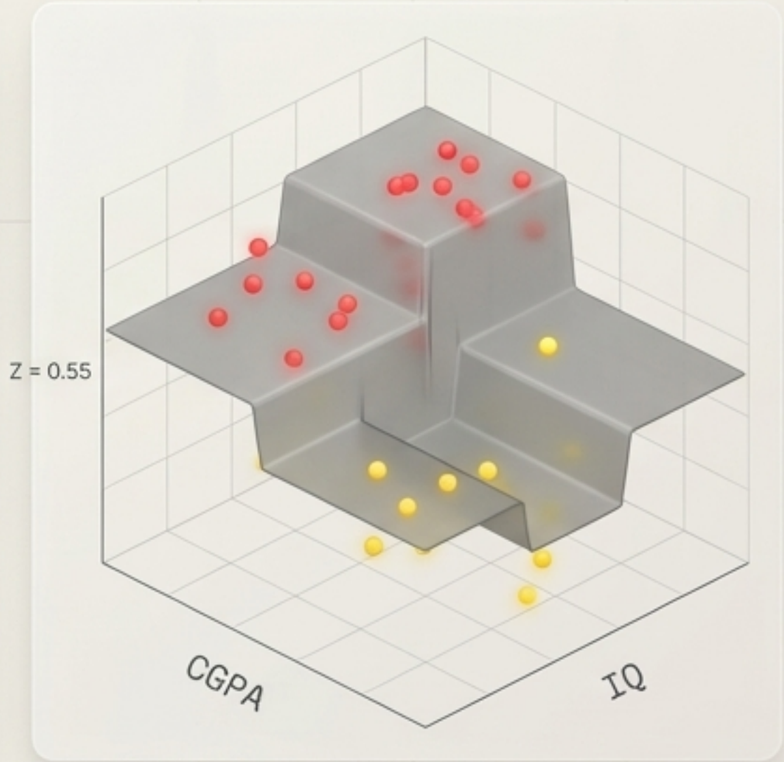
Visualising the Mathematics: The 3D Surface

If we plot our 2D features (CGPA, IQ) on the floor and the Target (0 or 1) as elevation, we can watch the additive engine physically sculpt the decision boundary.

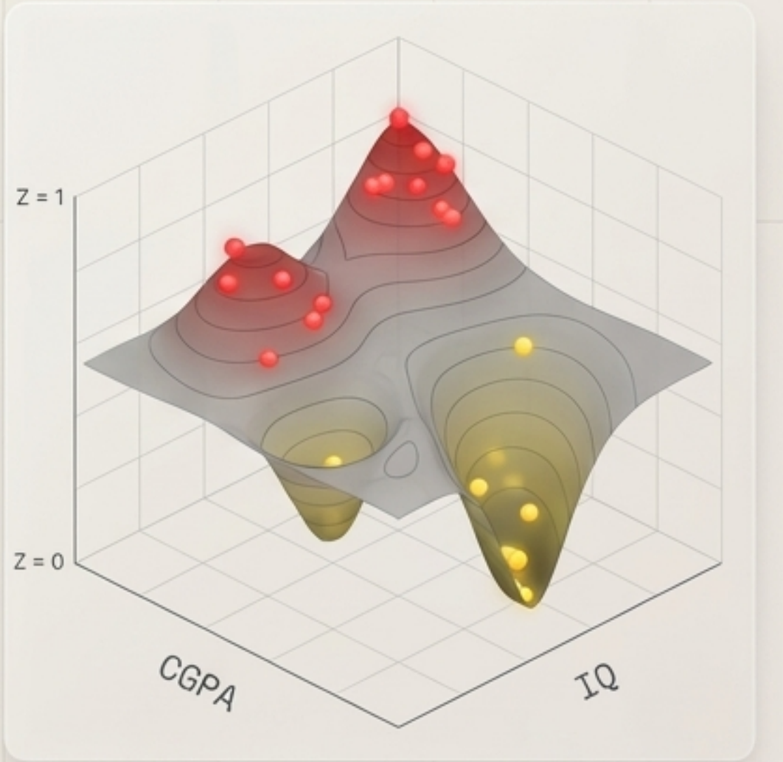
Stage 1



Stage 2



Final Stage



NotebookLM

The Architecture of Gradient Boosting

The power of this algorithm does not come from a single complex equation, but from the disciplined, iterative execution of three foundational rules.

1. The Goal (Error Correction)

Every new structural layer focuses exclusively on the mathematical residuals left by its predecessor.

2. The Engine (Translation)

Calculations operate safely in the unbounded Log-Odds space, evaluating success in the bounded Probability space.

3. The Control (Shrinkage)

A precise Learning Rate throttles the impact of each new tree, guaranteeing robust, slow convergence rather than volatile overfitting.

NotebookLM